ATS API Specification

Day 16 – ATS API Design & Integration Planning

1. Overview

The ATS API provides a REST-based interface between the Zecpath AI resume screening modules and a backend application.

The purpose of the API layer is to expose existing resume-processing, ATS-scoring, candidate-ranking, and shortlisting functionality through structured HTTP endpoints.

The API is implemented using FastAPI and follows a versioned REST architecture.

Base API path:

```
/api/v1
```

The API supports the following major operations:

- Resume upload
- Resume parsing
- ATS candidate scoring
- Candidate ranking and shortlisting
- Asynchronous job-status retrieval
- API health monitoring

---

2. Objective

The main objectives of the ATS API are:

- Provide a standard interface for backend integration.
- Expose ATS AI functionality through REST endpoints.
- Define predictable request and response structures.
- Validate incoming candidate data.
- Support asynchronous resume-processing workflows.
- Provide standardized error handling.
- Maintain logging for important API operations.
- Keep AI modules independent from frontend/backend implementation details.

---

3. API Architecture

The API acts as an integration layer between backend systems and the existing ATS modules.

```
Frontend / Recruiter Interface
            ↓
      Backend System
            ↓
        ATS REST API
            ↓
  ┌──────────────────────────┐
  │ Resume Processing        │
  │ ATS Scoring              │
  │ Ranking                  │
  │ Shortlisting             │
  └──────────────────────────┘
            ↓
       Structured Result
```

---

4. Technology

The API implementation uses:

```
Framework: FastAPI
Language: Python
Validation: Pydantic
API Style: REST
Data Format: JSON
API Version: v1
```

FastAPI was selected because it provides request validation, structured API development, asynchronous support, and automatic API documentation capabilities.

---

5. API Endpoints

5.1 Health Check

Method

```
GET
```

Endpoint

```
/api/v1/health
```

Purpose

Checks whether the ATS API service is operational.

Example Response

```
{
    "success": true,
    "message": "ATS API is operational.",
    "data": {
        "service": "Zecpath ATS API",
        "status": "healthy"
    }
}
```

5.2 Resume Upload

Method

```
POST
```

Endpoint

```
/api/v1/resumes/upload
```

Purpose

Accepts a candidate resume file and creates a processing job.

Supported formats:

```
PDF
DOCX
```

The uploaded resume receives a unique resume identifier and processing job identifier.

Example Response

```
{
    "success": true,
    "message": "Resume uploaded successfully.",
    "data": {
```

```
        "resume_id": "generated-resume-id",
        "job_id": "generated-job-id",
        "status": "QUEUED"
    }
}
```

Unsupported file formats return an appropriate client error.

5.3 Resume Parsing

Method

```
POST
```

Endpoint

```
/api/v1/resumes/parse
```

Purpose

Provides an API contract for passing resume text into the resume-processing pipeline.

Example Request

```
{
    "resume_text": "Data Scientist with Python, SQL and Machine Learning
experience."
}
```

Example Response

```
{
    "success": true,
    "message": "Resume parsing completed.",
    "data": {
        "text_length": 68,
        "status": "parsed",
        "integration": "Existing resume parser pipeline can be connected here."
    }
}
```

The endpoint provides the integration point for connecting the existing resume parsing modules to the backend API.

5.4 Candidate ATS Scoring

Method

```
POST
```

Endpoint

```
/api/v1/scoring
```

Purpose

Calculates an overall ATS candidate score using the existing ATS Scoring Engine.

The scoring API accepts:

- Job role
- Skill match score
- Experience relevance score
- Education alignment score
- Semantic similarity score

Example Request

```
{
    "job_role": "Data Scientist",
    "scores": {
        "skill_match": 90,
        "experience_relevance": 80,
        "education_alignment": 75,
        "semantic_similarity": 85
    }
}
```

Tested Response

```
{
    "success": true,
    "message": "Candidate scoring completed.",
    "data": {
```

```
        "job_role": "Data Scientist",
        "overall_score": 85.0,
        "recommendation": "Strong Candidate",
        "score_breakdown": {
            "skill_match": 90.0,
            "experience_relevance": 80.0,
            "education_alignment": 75.0,
            "semantic_similarity": 85.0
        }
    }
}
```

5.5 Candidate Shortlisting

Method

```
POST
```

Endpoint

```
/api/v1/shortlisting
```

Purpose

Accepts candidate information and ATS scores and connects the API layer with the candidate shortlisting engine.

Example Request


```
{
    "candidates": [
        {
            "candidate_id": "C001",
            "name": "Candidate A",
            "score": 91
        },
        {
            "candidate_id": "C002",
            "name": "Candidate B",
            "score": 78
        },
        {
            "candidate_id": "C003",
            "name": "Candidate C",
            "score": 56
```

```
        }
    ]
}
```

The candidate shortlisting engine then assigns candidates to appropriate decision zones according to the configured thresholds.

Possible decisions include:

```
Shortlisted
Review
Rejected
```

5.6 Asynchronous Job Status

Method

```
GET
```

Endpoint

```
/api/v1/jobs/{job_id}
```

Purpose

Retrieves the current status of a resume-processing job.

Supported job states are:

```
QUEUED
PROCESSING
COMPLETED
FAILED
```

Example Response

```
{
    "success": true,
    "message": "Job status retrieved.",
    "data": {
        "job_id": "example-job-id",
```

```
        "job_type": "RESUME_PROCESSING",
        "status": "COMPLETED",
        "result": {},
        "error": null
    }
}
```

6. Asynchronous Processing

Resume processing may involve operations that take longer than a normal API request.

The Day 16 implementation therefore demonstrates an asynchronous job lifecycle using FastAPI background tasks.

```
Resume Uploaded
      ↓
Job Created
      ↓
QUEUED
      ↓
PROCESSING
      ↓
COMPLETED
```

If processing fails:

```
PROCESSING
     ↓
FAILED
```

The client can retrieve the job status using:

```
GET /api/v1/jobs/{job_id}
```

The current implementation uses temporary in-memory job storage for development and testing.

A production implementation can later replace this with persistent storage and a dedicated job-processing infrastructure.

7. Error Handling Standards

The API follows structured error handling.

Typical HTTP responses include:

Status	Meaning
200	Successful request
400	Invalid request or unsupported input
404	Requested resource not found
422	Request validation failure
500	Internal processing error

Example structured error:

```
{
    "success": false,
    "error": {
        "code": "RESOURCE_NOT_FOUND",
        "message": "Requested job was not found."
    }
}
```

---

8. Input Validation

Pydantic schemas are used to validate API input.

Candidate evaluation scores must remain within:

```
0 – 100
```

For example, a score such as:

```
150
```

is rejected with HTTP:

```
422 Unprocessable Entity
```

This prevents invalid scoring information from reaching the ATS Scoring Engine.

---

9. Logging Standards

The API uses the existing project logging utility.

Important events logged include:

- API health checks
- Resume upload requests
- Processing job creation
- Resume-processing status
- Scoring requests
- Successful scoring
- Shortlisting requests
- Successful shortlisting
- Processing failures
- Missing resources

Raw resume contents and unnecessary personal information should not be written to application logs.

---

10. Testing

The API implementation was tested using FastAPI's test client.

The following tests completed successfully:

```
Health endpoint test passed.
Resume parsing API test passed.
ATS scoring API test passed.
Candidate shortlisting API test passed.
Validation handling test passed.
Job error handling test passed.
```

Final result:

```
All ATS API Design and Integration tests passed successfully!
```

---

11. Conclusion

The Day 16 ATS API provides a structured REST interface for integrating the resume screening system with backend applications.

The API defines endpoints for resume upload, parsing, ATS scoring, candidate shortlisting, asynchronous processing status, and health monitoring.

Standard request contracts, validation, error handling, logging, and asynchronous job handling establish a clear foundation for future backend and recruiter-interface integration.